Illini Portal Fit Engine

A transfer-portal recruiting target board for Illinois Men's Basketball

Author: Mannat Talati · **Built for:** Illinois Men's Basketball Analytics Internship **Live app:** *(Streamlit link, see README)* · **Code:** *(GitHub link, see README)*

---

1. The problem

Roster construction in college basketball now runs through the transfer portal. Every spring, 1,500+ Division I players enter, and a staff has a few weeks to decide who to pursue. Illinois under Brad Underwood has been one of the most aggressive portal and international programs in the country, so this isn't a side project for the staff. It is the roster.

The hard part isn't finding *good* players; it's finding good players who **(a)** fill a specific hole the roster just opened, **(b)** fit how Illinois actually plays, and **(c)** are realistic to land. Doing that by eye across all of Division I is slow, and it leans toward the names you already know.

This tool turns that triage into a ranked, explainable board. It profiles Illinois's roster, detects what's leaving, and scores every eligible D-I player on a transparent **Fit Score**, so the staff can start the portal window with a shortlist instead of a blank page.

2. The data, and how I sourced it

Everything is public BartTorvik data, pulled live over plain HTTP. No scraping tricks, no paywalled sources like KenPom.

Source	Endpoint	What it gives
Player advanced stats	`getadvstats.php?year=YYYY&csv=1`	~5,000 players/season, 67 columns: archetype role, usage, efficiency, shooting splits, BPM/OBPM/DBPM, rebounding, steals/blocks, class, height, hometown
Team ratings	`YYYY_team_results.json`	365 teams: adjusted offense/defense, Barthag power rating, tempo

The player feed ships with no header row, so I reverse-engineered the column layout and then checked it against the data itself before trusting a single number:

- shooting splits reconcile (FTM ÷ FTA = FT%, etc.),
- `BPM == OBPM + DBPM` holds for 100% of rows,
- `total rebounds == offensive + defensive` per game holds for 100% of rows.

Columns I couldn't verify are named `extra_NN` and go unused on purpose, so every figure the tool reports is one I can explain on a whiteboard.

A real snapshot is committed to the repo, so the app loads instantly for a reviewer and the results are reproducible. A "Refresh from BartTorvik" button re-pulls live data on demand.

3. How the Fit Score works

For the current (2025-26) season, the engine reads Illinois as: 28-9, Barthag #6, the #2 offense in the country (adjOE 131.8), #20 defense, and, notably, the #289 tempo. That last number matters. Illinois is an *elite half-court* team, not a run-and-gun team, so the model rewards skill and shot-making over raw transition athleticism. The profile is derived from data, not assumed.

It then assumes seniors graduate (editable in the app). For 2025-26 that's lead guard **Kylan Boswell**, stretch big **Ben Humrichous**, and **AJ Redd**. It measures what walks out the door with them: a chunk of the team's playmaking, perimeter defense, and outside shooting.

Each eligible candidate (non-Illinois, has remaining eligibility, cleared minimum minutes/games) gets a **0-100 Fit Score**, a weighted blend of four parts that are each shown separately:

1. **Production**: proven value from BPM, porpag, and minutes load.
2. **Role fit**: does their position fill a *depleted* Illinois spot (guard / wing / big)?
3. **System fit**: do their strengths match the specific skills Illinois must replace? The weights here come straight from the departure analysis (lose 24% of your made threes and shooting gets weighted up).
4. **Attainability**: a realism lens from the player's current team strength, shown as a plain label: *High-major proven*, *Mid-major riser*, *Low-major standout*.

Every component is percentile-ranked within the candidate pool, so a score answers the question a coach actually asks: *"compared to the other realistic options, how good is this?"* All four weights, the departure list, and the need weights are tunable live in the app.

Example output (default Illinois needs, 2025-26):

Fit	Player	Team	Role	Why
86.8	Jason Rivera-Torres	Monmouth	Wing F	3.3% steal rate; 1.5 3PM/g; +5.5 BPM, perimeter D + shooting; mid-major riser
85.3	Tyler Tanner	Vanderbilt	Scoring PG	5.1 ast/g (29 AST%); +10.2 BPM, playmaking; high-major proven
85.1	Mason Falslev	Utah St.	Combo G	3.6% steal rate; 3.1 ast/g; +8.8 BPM, perimeter D + playmaking

Each row carries an auto-generated scouting note, and the **Player Detail** view breaks the score into its four bars next to the player's full statistical profile.

4. Three tools that make the board a scouting product

A ranked list is a start. A staff also needs to know what the numbers mean at our level, who a player reminds us of, and have something they can print and hand to a coach. Three additions do that, and the first two are calibrated on real history rather than assumed.

4a. Big Ten Translation: what a box line becomes at Illinois's level

A 20-and-8 against low-major defenses is not a 20-and-8 in the Big Ten, because box-score counting stats aren't opponent-adjusted. To make gaudy lines honest, I learned the deflation from data: I matched the same player across consecutive seasons using BartTorvik's stable player id, found everyone who changed competition level, and measured how much of each stat survived the jump. The calibration set is 990 real cross-season transfers.

For each stat I fit a one-parameter retention line, anchored so a zero-gap move means no change (`ratio = 1 + slope × level_gap`), where the gap is the Barthag distance between the player's team and Illinois (Barthag 0.968). The result is the scouting lesson you'd hope for:

Stat	Survives a jump?	Keeps, at a typical mid-major→Illinois jump
Points / g	deflates most	~78%
Rebounds / g	deflates	~84%
Assists / g	deflates	~86%
Usage%	deflates mildly	~88%
Made 3s / g	travels well	~90%
TS% (efficiency)	**holds (slope ≈ 0)**	**~101%**

So efficiency and made-shooting travel; raw volume deflates. And the deflation scales with how big a leap the player is making. A 21.4 ppg scorer in the SWAC projects to ~6.5 ppg in the Big Ten ("major jump, high variance"), while a high-major star at BYU keeps ~93% of his scoring. The board shows each candidate's raw line next to its projection, a **Keep%**, and a level-jump risk label, so a 25-ppg low-major name can't masquerade as a 25-ppg Big Ten name.

> Note on double-counting: the Fit Score's *production* component already uses opponent-**adjusted** inputs (BPM, adjusted ratings), so the translation model leaves the score alone and instead translates the raw box stats coaches read directly. That keeps the two numbers separate and both honest.

4b. Player comps: "who does he play like?"

During portal season a staff constantly asks two questions: *a starter just left, which available transfers replace his style?* and *we like this target, who does he remind us of?* Both are nearest-neighbor problems in a *style space*: each player becomes a vector of role/skill rates (usage, shot diet, playmaking, ball security, rebounding, rim protection, perimeter activity, size, efficiency), z-scored against the D-I rotation baseline so every axis is comparable. Distance is plain Euclidean, no learned weights, reported as a 0-100 similarity. The **Program & Needs** page turns a departing Illini into a ranked list of transfer-eligible look-alikes, and every target carries its closest high-major comps.

4c. Visual scouting cards + printable PDF

Coaches read cards, not 60-column CSVs. Each target renders a one-page card: a percentile radar vs the Big Ten rotation (every spoke is "how would this skill rank in our league?"), the raw → Big Ten projected box line, a shot diet (rim / mid / three share and accuracy), and the player's comps. The

whole card exports to a self-contained one-page PDF the staff can print for the board.

5. Four more tools for the actual decision

The three tools above make the board readable. These four help with the decision itself: they answer the follow-up questions a staff asks the moment a name rises to the top. The first two also turn the model's honest limitations into something useful, and all four reuse the same public, cross-season BartTorvik data (the same player matched across seasons by stable `pid`).

5a. Outcome-backed projections: real precedent behind every number

A projection a staff can't trace is a projection a staff won't trust. So for any target, the engine pulls the real historical transfers who started from the most similar profile and made a similar-size level jump (matched in the same z-scored style space, penalized when the jump sizes differ), then reports what actually happened to their scoring the next season. A 21.7-ppg ASun scorer jumping to Illinois surfaces names like Frankie Fidler (20.1 → 7.0), Chaz Lanier (19.6 → 18.0), and, fittingly, Ben Humrichous (14.8 → 7.6). That yields a band: *"6 comparable real transfers kept 38–62% of their scoring, median 46% → ~8–14 pts at our level."* It replaces a single deflated number with an empirical distribution drawn from the same 600+ up-transfers, so the staff sees the range of outcomes instead of false precision. The named precedent is what makes the translation defensible in a room.

5b. Development trajectories: who is rising, not just who was good

A single season underrates an ascending player and overrates a one-year spike. By tracking the same player across seasons (stable `pid`), the engine measures each returning rotation player's year-over-year move in BPM, usage, efficiency, scoring, and minutes, and labels him *Breakout* (a bigger role that *held* efficiency, still underclass-eligible), *Ascending*, *Steady*, or *Declining*. To keep it honest, the trajectory only counts players who held a real rotation role in *both* seasons. A +20 BPM "jump" off a five-minute freshman year is noise, not growth. The board carries a ΔBPM column and a dedicated **risers** view, and Player Detail shows the full last-year → this-year line. A sophomore who jumped from –0.8 to +4.0 BPM at a high-major reads as the breakout he is, not as an average-looking single-season line.

5c. Head-to-head comparison: three targets, one screen

When two or three names cluster at the top, the staff compares them directly: overlaid percentile radars (vs the Big Ten), every Fit component, the projected box line, and shooting/defense rates side by side, each with its one-line scouting note. The decision a coach actually makes (which of these wings?) gets its own screen instead of a mental juggle across rows.

5d. Post-portal depth chart: tie every target to a roster hole

Finally, the engine projects the roster *after* the marked departures into a Guard / Wing / Big depth chart against a target rotation (4 / 2 / 3), shows how many spots are open at each position, and lets the staff slot board targets into the holes and watch the rotation fill in. It closes the loop from "good player" back to "fills *our* November rotation."

6. How I built it

- **Language:** Python 3.
- **Data / model:** `pandas` and `numpy`. A small, readable package (`illini_fit/`) split into `schema` (validated column map), `fetch` (cache + live refresh), `profile` (team + roster), `needs` (departure-driven gap detection), `fit_score` (the scoring model), `translation` (the Big Ten retention model), `similarity` (the style-comp engine), `precedent` (outcome-backed comparable transfers), `trajectory` (multi-year development), and `scouting` (radar + PDF cards).
- **App:** `streamlit`, an interactive Illini-themed web app: filters, weight sliders, a ranked board with projected lines and a risers view, a post-portal depth chart, a head-to-head compare screen, player detail with development trajectory plus real precedents plus scouting card, and CSV + PDF export. `matplotlib` draws the radars and `xhtml2pdf` renders the printable card.
- **Quality:** every module ships with self-checks you run as `python -m illini_fit.<module>`: data row-count and Illinois-parse assertions; a test that a shooting-weighted board surfaces more shooters and that scores stay in 0-100; that scoring deflates and efficiency holds in the translation model; that a player is his own closest comp; that the precedent cohort is large and its kept-% band is ordered; that a big BPM jump reads as a riser. The app itself is verified headlessly with Streamlit's `AppTest`, including the compare and depth-chart interactions.
- **Deploy:** GitHub + Streamlit Community Cloud (one-click, free) for a shareable live link.

7. Why it's useful to a GM / coaching staff

- **It starts the portal window with a shortlist, not a spreadsheet.** Day one of the portal, the staff has a ranked, position-aware board instead of 1,500 names.
- **It's tied to *this* roster's needs.** The board re-ranks the moment you mark a player as leaving. The night a starter enters the portal, you see who replaces *his* production, not just "good players."
- **It's explainable to non-analysts.** Every score breaks down into production / role / system / attainability with a one-line rationale. That holds up in a staff meeting and with a head coach who wants the *why*, not a black box.
- **It encodes Illinois's identity from data.** Because system fit is derived from Illinois's actual profile (elite half-court offense, slow tempo, shooting), it won't recommend a player who is good in the abstract but wrong for how Illinois plays.
- **It translates production to *our* level, and shows the receipts.** The Big Ten projection stops a 25-ppg low-major name from masquerading as a 25-ppg Big Ten name, and the precedent engine

backs every projection with real named transfers who made the same jump plus a band of what they actually kept. Misreading raw portal stats is the most common mistake in this work, and here the evidence is attached.

- **It finds risers, not just last year's stars.** Multi-year trajectories flag the ascending sophomore the box score underrates and fade the one-year spike. That's exactly the edge a development program like Illinois wants.
- **It speaks the staff's language and closes the loop.** "Plays like" comps frame an unknown name in terms of players the room already knows; the compare screen settles which of these three wings; the depth chart drops a target into an open rotation spot; and the one-page PDF card is something a coach can print and carry.
- **It's tunable on the fly.** A coach who wants a defensive guard over a scoring one just moves a slider, and the board updates in real time.

In practice the staff intersects this board with the live portal list (which they have, and which isn't a clean public feed): the engine ranks the full pool by fit, and the staff filters to who's actually available. That mirrors how an analytics staffer already works; this just does the heavy ranking in seconds.

8. Honest limitations and next steps

- **Portal availability isn't a clean public dataset**, so the engine ranks the full pool and is meant to be filtered to the live portal list. A natural v2 is to ingest a portal feed and auto-filter.
- **The Fit Score still reads one season**, though the development trajectories now add multi-year context alongside it. A candidate with no prior D-I season (true freshmen, some internationals) has no trajectory, and play-type data (Synergy/Hudl, if licensed) would sharpen role fit further.
- **The translation model is a population average.** That's exactly why the precedent engine sits next to it: instead of trusting a single deflated number, the staff sees the band of what real comparable transfers actually kept. An individual can still beat or miss his comps, so the engine surfaces the range and a level-jump risk label rather than false precision.
- **Fit weights are a reasonable default, not gospel**, which is why they're exposed as sliders for the staff to own.
- **No NIL/cost modeling.** A real GM board would layer in budget, and that data isn't public.

All data is publicly sourced from BartTorvik. Built as an independent analytics project for the Illinois Men's Basketball Analytics Internship.